Xerox Shop Online Ordering System

Final Scope & Detailed Technical Blueprint

Java 21 • Spring Boot • Spring Security • React.js • MySQL

Included in this document

- Eight selected features with requirements, database changes, APIs, security controls, and implementation notes.
- Multi-shop architecture and tenant isolation.
- Database table catalogue and recommended constraints.
- REST API inventory and package structure.
- Phased implementation roadmap and release-readiness checklist.
- Important payment, file privacy, pickup capacity, OCR, and duplicate-print safety rules.

Purpose: a build-ready planning document. Exact implementation details may be adjusted to fit the existing project schema and chosen service providers.

1. Executive Summary

The system lets customers select a Xerox shop, upload documents, choose specific pages and print settings, select a pickup slot, pay online, and track order progress. Shop admins review paid orders and explicitly initiate printing through an authorized shop-local Print Agent. The system supports multiple shops, shop-specific pricing and pickup capacity, notifications, financial/inventory reports, OCR, and a customer-support chatbot.

Core rule: successful payment does not automatically mean a document is printed. An authorized admin must review the order and deliberately initiate a print job.

Technology Stack

Layer	Recommended technology
Frontend	React.js with Vite; responsive customer portal and admin dashboards
Backend	Java 21, Spring Boot 3.x, Spring Security, Spring Data JPA/Hibernate, Bean Validation
Database	MySQL 8.x; transactional reservations, unique constraints, indexes, versioned migrations
Authentication	Spring Security with secure session or short-lived access tokens and refresh strategy; role and shop membership checks
File storage	Private filesystem/object storage outside public webroot; optional short-lived signed links after authorization
Printing	Shop-local Print Agent with authenticated outbound communication to backend
Integrations	Payment gateway, WhatsApp/SMS provider, OCR engine, optional LLM/chatbot provider
Operations	HTTPS, secrets management, logs/metrics, backups, monitoring, migration tool

2. User Roles and Main Workflow

- Customer: browse/select shop, register/login, upload files, preview pages, choose settings and pickup time, pay, track status, receive notifications, request cancellation where policy allows.
- Shop admin: manage shop settings, pricing, slots, orders, print queue, printers, inventory, expenses, reports, and shop-level notification settings.
- Platform admin: create/suspend shops, manage memberships and platform configuration, and access cross-shop reports only under explicit authorization.
- Background workers: process uploads/scanning, OCR, notification outbox, payment events, retention deletion, and operational retries.

```
Customer → Select Shop → Upload → Preview/Settings → Price Quote → Pickup Slot → Payment Verification
→ Admin Review → Print Queue → Ready for Pickup → Collection
```

Order state, payment state, and print-job state must be separate. Example: an order may be PAID while its print job is still QUEUED. Store status transitions in an order history/audit trail.

3. Feature 1 — Specific Page Selection and Print Preview

Requirements

- Accept PDF and supported image formats; generate page count and preview thumbnails after safe validation.
- Support all pages, ranges such as 1-5, individual pages such as 2,4,7, and combinations such as 1-3,8.
- Allow copies, paper size, orientation, color/grayscale, simplex/duplex, and per-file settings.

- Recalculate server-side price whenever selected pages or print options change; show an estimate before checkout.
- Support multiple files in one order, with independent settings per file.

Database

Table	Key fields / purpose
order_files	id, shop_id, order_id, storage_key, original_filename, mime_type, file_size_bytes, page_count, sha256_hash, scan_status, created_at
print_settings	id, order_file_id, page_selection, copies, paper_size, orientation, color_mode, duplex_mode, pages_to_print, created_at
orders	subtotal, tax_amount, total_amount, currency, pricing_version; final amount must be calculated by server

APIs

Method	Endpoint	Purpose
POST	/api/v1/shops/{shopId}/uploads	Upload document
GET	/api/v1/files/{fileId}/preview	Authorized preview
GET	/api/v1/files/{fileId}/metadata	Page count and metadata
POST	/api/v1/shops/{shopId}/price-quotes	Calculate a quote
PUT	/api/v1/orders/{orderId}/print-settings	Save/update settings

Security and validation

- Validate page ranges on the server against the parsed page count; reject zero, negative, out-of-range, malformed, or excessive selections.
- Do not trust a total sent by React. Calculate and persist the server-side quote, including the applicable pricing version.
- Require authorization for every preview/download request; never expose permanent public document URLs.
- Warn that preview rendering can differ from physical printer output because of fonts, margins, and device capabilities.

4. Feature 2 — Pickup Slot Capacity Management

Requirements

- Each shop manages opening hours, slot start/end times, closures, holidays, and capacity.
- Capacity may be limited by order count, page count, or a configurable workload estimate.
- Full slots are unavailable to new bookings; cancellation/rescheduling adjusts capacity according to policy.
- Pending payment reservations have a defined hold/expiry policy; expired reservations release capacity safely.

Database

Table	Key fields / purpose
pickup_slots	id, shop_id, start_time, end_time, capacity_orders, capacity_pages, reserved_orders, reserved_pages, status, version
slot_reservations	id, shop_id, slot_id, order_id, reserved_orders, reserved_pages, status, expires_at, created_at

Table	Key fields / purpose
orders	pickup_slot_id, pickup_date, pickup_status

APIs

Method	Endpoint
GET	/api/v1/shops/{shopId}/pickup-slots?date=YYYY-MM-DD
POST	/api/v1/shops/{shopId}/pickup-slots
PATCH	/api/v1/shops/{shopId}/pickup-slots/{slotId}
POST	/api/v1/orders/{orderId}/pickup-slot
POST	/api/v1/orders/{orderId}/reschedule

Concurrency controls

- Reserve capacity inside a database transaction using row locking (for example, SELECT ... FOR UPDATE) or a correctly implemented optimistic-locking strategy.
- Never rely only on the slot count displayed in React. Re-check capacity during reservation.
- Make reservation creation idempotent and release capacity exactly once when a reservation expires or is cancelled.
- Define what happens if payment succeeds after the reservation expires; reconcile the payment and offer a safe reschedule/refund path.

5. Feature 3 — Admin Print Queue and Duplicate Print Protection

Requirements

- Queue eligible orders by priority, pickup deadline, and submission time.
- Allow authorized admins to review, start, cancel, and retry jobs; show printer state, attempts, errors, and job history.
- Printing must require verified payment and an allowed order state.
- Use a shop-local Print Agent to communicate with local printers; do not promise silent browser printing.
- Provide an explicit manual review state when the printer may have printed but confirmation was lost.

Database

Table	Key fields / purpose
print_jobs	id, shop_id, order_id, order_file_id, idempotency_key, status, attempt_count, requested_by, printer_id, created_at, started_at, completed_at, error_code
printers	id, shop_id, name, printer_type, status, last_seen_at
print_job_events	id, print_job_id, event_type, actor_id, details, created_at

APIs

Method	Endpoint
GET	/api/v1/shops/{shopId}/print-queue
POST	/api/v1/shops/{shopId}/print-jobs

Method	Endpoint
POST	/api/v1/print-jobs/{jobId}/start
POST	/api/v1/print-jobs/{jobId}/retry
POST	/api/v1/print-jobs/{jobId}/cancel
GET	/api/v1/print-jobs/{jobId}/events

Duplicate protection and security

- Enforce a unique idempotency key at database level and make job creation transactional.
- Use a stable job identity for the order/file/settings combination and prevent duplicate active jobs.
- The Print Agent must authenticate, validate job identity, and report job events. Do not expose printer credentials to React.
- Physical printers cannot always guarantee exactly-once output: a lost acknowledgment can leave the result uncertain. Use UNKNOWN/NEEDS_REVIEW rather than blind automatic reprinting.
- Restrict queue actions to authorized members of the matching shop and log each state transition.

6. Feature 4 — Secure Upload and Private File Access

Requirements

- Allowlist formats such as PDF, JPG, JPEG, and PNG; enforce file-size, file-count, and total-upload limits.
- Validate extension, content signature, MIME type, and file parseability; scan for malware.
- Store files outside the public webroot or in private object storage with server-generated storage keys.
- Track quarantine/scanning state, retention deadlines, and deletion status.
- Keep an access audit trail for sensitive download and preview operations.

Database and APIs

Use order_files with shop_id, order_id, storage_key, original_filename, mime_type, file_size_bytes, sha256_hash, scan_status, retention_until, deleted_at. Add file_access_audit with file_id, shop_id, actor_id, action, timestamp, and outcome.

Method	Endpoint
POST	/api/v1/shops/{shopId}/uploads
GET	/api/v1/files/{fileId}/download
GET	/api/v1/files/{fileId}/preview
DELETE	/api/v1/files/{fileId}

Security

- Authorize ownership/shop membership on every access, not only at upload time.
- Never use the customer-supplied filename as a path; generate storage keys on the server.
- Keep quarantined files unavailable until scanning succeeds. Apply rate limits and upload quotas.
- Use short-lived signed URLs only after authorization when appropriate; do not log signed URLs or sensitive document contents.
- Publish a retention/deletion policy and ensure backup retention is considered when deleting documents.

7. Feature 5 — WhatsApp/SMS Notifications

Requirements

- Notify for order received, payment confirmed, printing, ready for pickup, cancellation, and refund status.
- Record consent and distinguish transactional from promotional messaging.
- Handle provider delivery callbacks, transient failures, retry/backoff, and duplicate event suppression.
- Use templates and a configurable provider integration rather than hard-coding provider-specific logic into order services.

Database

Table	Key fields / purpose
notifications	id, shop_id, order_id, user_id, channel, template_code, recipient_masked, provider_message_id, status, attempt_count, sent_at, error_code
notification_outbox	id, shop_id, event_type, aggregate_id, payload, status, created_at, processed_at

APIs

Method	Endpoint
GET	/api/v1/orders/{orderId}/notifications
POST	/api/v1/shops/{shopId}/notification-settings
POST	/api/v1/webhooks/notifications/{provider}

Reliability and security

- Commit the business event and outbox record in the same database transaction; a background worker sends messages later.
- Store API secrets server-side, verify provider callback signatures, and suppress duplicate callbacks.
- Avoid sending document contents, passwords, or payment secrets in messages.
- Delivery status is not proof that the recipient read the message.

8. Feature 6 — Sales, Expense, and Inventory Reports

Requirements

- Daily/weekly/monthly sales and date-filtered shop reports.
- Show gross sales, discounts, taxes where applicable, refunds, net collected payments, and outstanding amounts separately.
- Record expenses such as paper, toner, maintenance, rent, and other shop costs.
- Track inventory purchases, stock adjustments, reorder thresholds, and consumption linked to verified print jobs.
- Support CSV exports with the same authorization and filters as the report view.

Database

Table	Key fields / purpose
expenses	id, shop_id, category_id, amount DECIMAL, currency, expense_date, description, created_by

Table	Key fields / purpose
expense_categories	id, shop_id, name, is_active
inventory_items	id, shop_id, name, unit, current_quantity, reorder_level, average_unit_cost
inventory_transactions	id, shop_id, inventory_item_id, transaction_type, quantity, unit_cost, reference_type, reference_id, created_by, created_at
payments/refunds	Provider references, captured amount, refund amount/status, verified timestamps, currency

APIs

Method	Endpoint
GET	/api/v1/shops/{shopId}/reports/sales?from=&to;=
GET	/api/v1/shops/{shopId}/reports/expenses?from=&to;=
GET	/api/v1/shops/{shopId}/reports/inventory
POST	/api/v1/shops/{shopId}/expenses
POST	/api/v1/shops/{shopId}/inventory/transactions
GET	/api/v1/shops/{shopId}/reports/export?type=sales&from;=&to;=

Financial correctness

- Use DECIMAL for money, never binary floating-point. Store currency and time zone context.
- Use verified payment and refund records for reports; do not treat a front-end redirect as proof of payment.
- Prefer immutable financial transactions and reversal/adjustment records instead of deleting financial history.
- Distinguish actual inventory consumption from estimated consumption. Reconcile print counts and stock movements.
- Confirm local tax/accounting requirements with a qualified accountant.

9. Feature 7 — AI Chatbot and Document OCR

Requirements

- Chatbot answers workflow questions using approved shop policy and pricing data; authenticated order lookup must enforce ownership.
- OCR extracts text from scanned PDFs/images and presents results for customer review.
- OCR jobs run asynchronously and expose processing status and errors.
- Apply provider quotas, maximum pages/file size, cost limits, timeouts, and retry policy.
- Do not let model output bypass authorization, set final prices, or trigger printing/payment actions without validated application workflows.

Database

Table	Key fields / purpose
ocr_jobs	id, shop_id, order_file_id, status, engine, language, result_storage_key, error_code, completed_at
chat_sessions	id, shop_id, user_id, created_at, closed_at
chat_messages	id, session_id, role, message_text, created_at, safety_label

APIs

Method	Endpoint
POST	/api/v1/files/{fileId}/ocr
GET	/api/v1/ocr-jobs/{jobId}
GET	/api/v1/files/{fileId}/ocr-result
POST	/api/v1/chat/sessions
POST	/api/v1/chat/sessions/{sessionId}/messages

Security, privacy, and AI safety

- Treat OCR text and uploaded document contents as untrusted data, not instructions.
- Show OCR output as unverified text; users must review important values.
- Use authoritative application APIs for price, order status, pickup slots, and permissions.
- Check provider retention and training policies before sending sensitive documents to an external AI service; obtain appropriate consent and disclose processing.
- Keep unnecessary personal data out of chat history, encrypt sensitive data where appropriate, and define retention/deletion rules.

10. Feature 8 — Multiple Xerox Shops

Requirements

- Each shop has its own profile, address, opening hours, pricing, pickup slots, printers, inventory, expenses, and reporting.
- Customers choose a shop; future discovery can use area/location and shop availability.
- Shop admins see only their shop's records. Platform admins have a separate explicitly authorized role.
- Users may belong to more than one shop through membership records.
- Suspended/closed shops cannot accept new orders, while existing orders remain accessible for completion/refund.

Database

Table	Key fields / purpose
shops	id, name, slug, address, phone, timezone, status, created_at
shop_members	id, shop_id, user_id, role, status; unique membership per user/shop
Tenant-owned tables	shop_id, suitable foreign keys, indexes, and tenant-aware uniqueness constraints

APIs

Method	Endpoint / permission
GET	/api/v1/shops — public/limited shop discovery
GET	/api/v1/shops/{shopId} — shop details
POST	/api/v1/shops — platform admin only
PATCH	/api/v1/shops/{shopId} — authorized shop/platform admin
POST	/api/v1/shops/{shopId}/members — platform or authorized shop admin

Method	Endpoint / permission
GET	/api/v1/shops/{shopId}/pricing — shop pricing

Tenant isolation rules

- Treat shop_id from the client as a requested scope, never as proof of authorization.
- Resolve the user's shop membership and role on every protected request.
- Enforce tenant isolation in service/repository access and file storage authorization, not just the frontend.
- Use tenant-aware unique constraints where necessary (for example, shop_id + slug/code).
- Audit cross-shop/platform operations and test for insecure direct object reference (IDOR) vulnerabilities.

11. Final Database Catalogue

Domain	Tables
Identity and tenants	users, roles/role mappings, shops, shop_members
Orders and print options	orders, order_files, print_settings, order_status_history
Pricing and scheduling	print_pricing, pickup_slots, slot_reservations
Payments	payments, refunds, payment_webhook_events
Printing	printers, print_jobs, print_job_events
Files and OCR	file_access_audit, ocr_jobs
Notifications	notifications, notification_outbox
Finance and inventory	expenses, expense_categories, inventory_items, inventory_transactions
AI assistant	chat_sessions, chat_messages
Audit and operations	audit_logs

Recommended database rules

- Use foreign keys and indexes based on actual query patterns; common candidates include (shop_id, status, created_at).
- Use DECIMAL for monetary values; use explicit currency codes.
- Use UTC timestamps for persistence and the shop's timezone for presentation/scheduling.
- Use unique constraints for idempotency keys, provider event IDs, and membership uniqueness.
- Keep order status, payment status, and print-job status separate.
- Use versioned migrations (such as Flyway or Liquibase); avoid destructive schema replacement on a live database.
- Use soft-delete or immutable history where auditability is important; apply secure retention deletion for documents.

12. REST API Conventions

- Prefix APIs with /api/v1 and return consistent JSON error bodies with a stable error code, safe message, and correlation/request ID.
- Use DTOs for requests/responses; do not expose JPA entities directly.
- Validate input using Bean Validation and enforce state transitions in services.
- Use pagination/filtering for order queues, files, notifications, and audit records.

- Return 401 for unauthenticated requests, 403 for unauthorized scope, 404 when appropriate to avoid leaking object existence, 409 for conflicts such as full slots or invalid state transitions, and 422/400 for invalid input according to the API convention.
- Verify payment webhook signatures and reconcile amount, currency, provider reference, and order before marking payment successful.
- Apply rate limits to login, upload, quote, OCR, chatbot, and notification endpoints.

13. Suggested Spring Boot Package Structure

```
com.xerox
■■■ auth
■■■ users
■■■ shops
■■■ orders
■■■ files
■■■ printsettings
■■■ pricing
■■■ pickup
■■■ payments
■■■ printing
■■■ notifications
■■■ reports
■■■ inventory
■■■ ocr
■■■ chatbot
■■■ audit
■■■ common
    ■■■ config
    ■■■ exception
    ■■■ security
    ■■■ validation
```

Within each feature, add controller, service, repository, entity, DTO, and mapper classes as needed. Keep business rules in services, not in React or controllers.

14. Implementation Order and Milestones

Phase	Work	Exit criteria
1	Multi-shop foundation: shops, members, roles, tenant checks, migrations, audit	Shop A cannot access Shop B data
2	Secure upload, private storage, scan/quarantine, metadata, preview	Unsafe files blocked; authorized preview works
3	Page selection, settings, pricing quotes	Server validates pages and price
4	Pickup slots and capacity reservations	Concurrent last-slot booking is safe
5	Print queue, idempotency, Print Agent, retry/manual review	Unpaid orders blocked; duplicate requests safe
6	Notification outbox and WhatsApp/SMS provider	Retries and duplicate callbacks handled
7	Sales, expenses, inventory and exports	Reports reconcile to payments/refunds
8	OCR and chatbot	OCR review, quotas, privacy and authorization tested
9	Integration tests, backups, monitoring, deployment	Critical release checklist passes

15. Testing and Release Checklist

Multi-shop security

- [ ] Shop A admin cannot view Shop B orders.
- [ ] Customer cannot access another customer's files.

- [ ] Unauthorized roles cannot invoke admin APIs.

Upload and printing

- [ ] Invalid or oversized upload is rejected.
- [ ] Invalid PDF page ranges are rejected.
- [ ] Repeated print request does not create a duplicate job.
- [ ] Unpaid order cannot be printed.

Pickup and payment

- [ ] Two customers cannot reserve the final slot simultaneously.
- [ ] Invalid payment webhook signature is rejected.
- [ ] Duplicate payment webhook does not duplicate financial records.

Notifications and reporting

- [ ] Notification retry does not create duplicate events.
- [ ] Refunds are reflected correctly in net sales.
- [ ] Inventory adjustments have an audit trail.

AI and OCR

- [ ] OCR output is treated as unverified.
- [ ] Chatbot cannot reveal another customer's order.
- [ ] AI usage is limited by shop/user and file/page quotas.

16. Deployment and Operational Requirements

- Use HTTPS for frontend/backend and authenticated, encrypted communication between backend and shop-local Print Agent.
- Store DB credentials, payment keys, WhatsApp/SMS credentials, and AI provider keys in environment secrets/secret manager; never commit them to Git.
- Use separate development, sandbox/test, and production configurations.
- Set up automated database backups and test restore procedures.
- Centralize application logs and metrics; avoid logging passwords, full documents, tokens, or sensitive message contents.
- Monitor payment webhook failures, stuck reservations, print jobs in UNKNOWN state, notification failures, OCR backlog, storage use, and database health.
- Define recovery procedures for payment mismatch, printer failure, duplicate job attempts, and shop closure.

17. Important Decisions to Finalize Before Production

- Payment gateway, refund policy, and when slot reservations expire.
- File size/page limits, supported formats, retention period, and deletion policy.
- Whether capacity is based on orders, pages, or estimated print workload.
- Print Agent installation model, supported operating systems/printers, and handling of uncertain print outcomes.
- WhatsApp/SMS provider, consent records, approved templates, and retry limits.

- OCR engine/languages and whether document content may be processed by an external provider.
- Chatbot provider, data retention settings, budget limits, and escalation to human support.
- Shop timezone, tax/accounting requirements, and platform-admin access policy.

18. Glossary

Term	Meaning
Tenant isolation	Controls that keep one shop's records inaccessible to other shops.
Idempotency key	A stable unique request identifier used to prevent duplicate business actions.
Outbox pattern	Persisting an event with a business transaction and processing it asynchronously.
OCR	Optical Character Recognition; extracting text from images or scanned documents.
Print Agent	A local service that receives authorized print jobs and communicates with a shop's printer.
Webhook	A provider-to-server callback that must be authenticated and safely processed.
RBAC	Role-Based Access Control; permissions based on roles and shop membership.
IDOR	Insecure Direct Object Reference; accessing another user's resource by changing an identifier.

Conclusion

Build the multi-shop security foundation first, then private document handling, print settings/pricing, pickup capacity, and controlled printing. Add messaging, reports/inventory, and AI after the core order/payment/printing workflow is reliable. The most important safeguards are server-side authorization, verified payment, transactional slot reservations, private file storage, tenant isolation, and safe handling of uncertain printer outcomes.

This blueprint is intentionally implementation-oriented, but endpoint names and columns should be reconciled with the existing schema before applying migrations. Test payment, file access, capacity, and print-job flows in a sandbox before production.